Structure d'un programme - fonctions

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Introduction aux Fonctions **4**

A. Définition et importance des fonctions 4

B. Syntaxe de base d'une fonction en Python 5

C. Exercice : Quiz 7

III - Portée des variables et fonctions avec paramètres **9**

A. Introduction 9

B. Portée des variables 9

C. Fonctions avec paramètres 10

D. Fonction récursive 10

E. Arguments positionnels vs arguments nommés 11

F. Exercice : Quiz 11

IV - Fonctions lambda et expressions lambda **14**

A. Introduction 14

B. Définition et utilisation 14

C. Exercice : Quiz 15

V - Fonctions Génératrices et Itérateurs **18**

A. Introduction 18

B. Fonctions génératrices avec « yield » 18

C. Nouveautés de Python 3.12 : Generator-based coroutines 19

VI - Essentiel **20**

VII - Auto-évaluation **21**

A. Exercice 21

B. Test 21

Solutions des exercices **24**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 heure

Prérequis : Aucun

Contexte

Dans le domaine de la programmation, les fonctions permettent de structurer le code de manière claire et efficace. Elles regroupent des instructions pour effectuer des tâches spécifiques, ce qui rend le code plus modulaire, réutilisable et facile à maintenir.

Nous allons ensemble :

- **Définir et comprendre l'importance des fonctions** : nous commencerons par expliquer ce qu'est une fonction, pourquoi elles sont cruciales dans la programmation, et comment elles peuvent améliorer la lisibilité et la réutilisabilité du code.
- **Maîtriser la syntaxe de base des fonctions en Python** : déclaration des fonctions, à utiliser l'instruction `return` pour renvoyer des valeurs, et à gérer les paramètres et les arguments des fonctions.

Tous droits réservés à STUDI - Reproduction interdite

II Introduction aux Fonctions

A. Définition et importance des fonctions

Az Définition

Une fonction est un bloc de code qui effectue une tâche particulière et qui peut être appelé à tout moment dans votre programme. Une fonction prend des entrées (appelées paramètres ou arguments), effectue des opérations sur ces entrées, et retourne éventuellement un résultat.

Imaginez une fonction comme une recette de cuisine. La recette (fonction) prend des ingrédients (paramètres), suit une série d'étapes (instructions) pour préparer un plat, et produit un résultat final (valeur de retour).

Prenons, par exemple, une fonction que nous allons appeler « *saluer* » : je ne donne aucun élément et j'affiche le mot « *Bonjour, tout le monde* ». En Python, on déclare une fonction via le mot-clé « *def* », qui permet de définir une fonction. Nous donnons ensuite le nom de la fonction, puis les paramètres entre parenthèses. Ensuite, on effectue un traitement ; dans notre cas, il s'agit juste d'afficher un message.

```
1 def saluer():
2     print("Bonjour, tout le monde!")
```

Si saluer a besoin d'argument, par exemple un nom pour afficher un message personnalisé, nous pouvons l'ajouter dans les parenthèses.

```
1 def saluer(nom):
2     print(f"Bonjour {nom} ")
```

Nous avons donné une variable, qui s'appelle « *nom* », mais qui peux être de type int ou encore str. Pour nous assurer que nous avons bien un nom, nous pouvons typer « *nom* »

```
1 def saluer(nom : str):
2     print(f"Bonjour {nom} ")
```

Enfin, je peux appeler la fonction dans mon code en donnant le nom, puis la valeur du paramètre.

```
1 def saluer(nom : str):
2     print(f"Bonjour {nom} ")
3 saluer("Arthur")
```

Les fonctions sont utiles pour plusieurs raisons :

- Réutilisabilité : une fonction peut être appelée plusieurs fois à différents endroits dans votre programme, évitant ainsi la répétition de code.
- Modularité : les fonctions permettent de diviser un programme complexe en sous-tâches plus simples et plus gérables.
- Lisibilité : en nommant des blocs de code, les fonctions rendent votre programme plus facile à comprendre.
- Maintenance : si vous devez modifier une tâche spécifique, vous n'avez qu'à mettre à jour la fonction correspondante, sans toucher au reste du code.

Par exemple, supposons que vous écriviez un programme pour gérer une bibliothèque et que vous devez fréquemment calculer des amendes pour les livres retournés en retard. Au lieu de répéter le même code de calcul d'amende partout, vous pouvez créer une fonction « *calculer_amende* » et l'appeler chaque fois que nécessaire.

Tous droits réservés à STUDI - Reproduction interdite

Rappel

La fonction « *calculer_amende* » se nommera ainsi car on utilise le snakecase comme convention de nommage. Les noms de fonctions doivent généralement toujours commencer par un verbe.

B. Syntaxe de base d'une fonction en Python

Az Définition

Pour déclarer une fonction en Python, nous utilisons le mot-clé « *def* » suivi du nom de la fonction et d'une paire de parenthèses. Le bloc de code de la fonction est indenté.

```
def saluer():
    print("Bonjour, tout le monde!")
```

Exemple

Dans cet exemple, nous avons défini une fonction appelée « *saluer* » qui imprime "Bonjour, tout le monde!" lorsqu'elle est appelée.

Pour appeler cette fonction, il suffit d'écrire son nom suivi de parenthèses :

```
saluer() # Affiche "Bonjour, tout le monde!"
```

Méthode

L'instruction « *return* » est utilisée pour renvoyer une valeur depuis une fonction. Cela signifie que vous pouvez utiliser le résultat de la fonction ailleurs dans votre programme.

```
def ajouter(a, b):
    return a + b
```

La fonction « *ajouter* » prend deux paramètres « *a* » et « *b* », et retourne leur somme. Vous pouvez appeler cette fonction et utiliser sa valeur de retour comme suit :

```
resultat = ajouter(5, 3)
print(resultat)  # Affiche 8
```

Méthode

Il est également possible de renvoyer plusieurs valeurs depuis une fonction en utilisant des tuples. Un tuple est une collection ordonnée d'éléments.

```
def obtenir_nom_complet(prenom, nom):
    nom_complet = f"{prenom} {nom}"
    longueur_nom = len(nom_complet)
    return nom_complet, longueur_nom
```

Tous droits réservés à STUDI - Reproduction interdite

La fonction « *obtenir_nom_complet* » prend deux paramètres « *prenom* » et « *nom* », et retourne à la fois le nom complet et la longueur de ce nom complet. Vous pouvez récupérer ces valeurs comme suit :

```
nom, longueur = obtenir_nom_complet("John", "Doe")
print(nom)        # Affiche "John Doe"
print(longueur)   # Affiche 8
```

Afin d'obtenir plus de visibilité sur les fonctions et se rendre compte du gain qu'ils vont vous procurer, voyons un cas concret avec notre ami José.

Exemple

José, le propriétaire d'un café, veut automatiser certaines tâches quotidiennes pour gagner du temps. Il a besoin de fonctions Python pour :

1. Calculer l'aire de ses tables pour commander des nappes.
2. Vérifier si le nombre de tables est pair ou impair pour des raisons logistiques.
3. Convertir la température de la cuisine de Celsius en Fahrenheit pour ses clients américains.

Pour le premier cas, nous allons devoir calculer l'aire de la table. Pour ce faire, la formule mathématique est : longueur multipliée par la largeur. Nous allons donc écrire une fonction python, qui a besoin de deux arguments : la longueur ainsi que la largeur de type float (car les données peuvent être décimales), ainsi, nous pourrons lui retourner l'aire de la table qui sera de type float également (un float multiplié par un float, donnera un type float).

```
def aire_rectangle(largeur : float, longueur : float) -> float:
    return largeur * longueur
# Utilisation de la fonction, pour l'exemple
largeur : float = 5
longueur : float = 10
aire : float = aire_rectangle(largeur, longueur)
print(f"L'aire du rectangle est: {aire}")  # Affiche "L'aire du rectangle est: 50"
```

Ensuite, José souhaite savoir si le nombre de tables est pair ou impair. Nous allons devoir potentiellement faire un traitement sur plusieurs tables, donc une fonction est encore la bienvenue ! Un nombre est pair si le reste de la division par 2 est 0 ; sinon, il est impair. Créons ensemble cette fonction.

```
def est_pair(nombre: int):
    return nombre % 2 == 0

# Utilisation de la fonction
nombre : int = 4
if est_pair(nombre):
    print(f"{nombre} est pair")  # Affiche "4 est pair"
else:
    print(f"{nombre} est impair")
```

Dans cet exemple, nous avons retourné vrai si le reste de la division de nombre avec 2 est 0, ou, false, si ce n'est pas le cas. L'appel de la fonction s'est fait dans une condition, c'est également possible et très pratique pour ne pas stocker un résultat dans une variable que nous n'aurions plus besoin par la suite.

Enfin, nous devons afficher la température du café en Fahrenheit pour ses clients américains. Nous, nous utilisons des degrés Celsius, nous devons multiplier notre température par 9/5, puis, ajouter 32 (car les Américains sont en degrés Fahrenheit)

Tous droits réservés à STUDI - Reproduction interdite

```python
def celsius_a_fahrenheit(celsius):
    return (celsius * 9/5) + 32

# Utilisation de la fonction
celsius = 25
fahrenheit = celsius_a_fahrenheit(celsius)
print(f"{celsius}°C est égal à {fahrenheit}°F")  # Affiche "25°C est égal à 77.0°F"
```

C. Exercice : Quiz

[solution n°1 p. 24]

Question 1

Qu'est-ce qu'une fonction en programmation ?

- Un type de variable qui peut stocker des valeurs
- Un bloc de code qui effectue une tâche particulière et peut être appelé à tout moment
- Un type de boucle qui permet d'itérer sur des éléments
- Une erreur de syntaxe dans un programme

Question 2

Pourquoi est-il important d'utiliser des fonctions dans un programme ?

- Pour augmenter la taille du code source
- Pour rendre le code plus compliqué
- Pour améliorer la réutilisabilité, la modularité et la lisibilité du code
- Pour réduire la vitesse d'exécution du programme

Question 3

Quelle est la syntaxe correcte pour déclarer une fonction en Python qui s'appelle « *saluer* » et affiche "Bonjour" ?**

- « *def saluer: print("Bonjour")* »
- « *function saluer() print("Bonjour")* »
- « *def saluer(): print("Bonjour")* »
- « *saluer() { print("Bonjour") }* »

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Que fait l'instruction « *return* » dans une fonction ?

- Elle arrête l'exécution du programme
- Elle imprime une valeur sur la console
- Elle renvoie une valeur depuis la fonction vers l'appelant
- Elle initialise une variable

Question 5

Quelle est la sortie du code suivant ?

```
1 def additionner(a, b):
2     return a + b
3
4 resultat = additionner(3, 4)
5 print(resultat)
```

- 3
- 4
- 7
- 12

Tous droits réservés à STUDI - Reproduction interdite

III Portée des variables et fonctions avec paramètres

A. Introduction

Dans cette partie, nous allons explorer la portée des variables, c'est-à-dire l'étendue dans laquelle une variable est accessible. Nous apprendrons également à définir des fonctions avec des paramètres, en couvrant les paramètres obligatoires et optionnels, ainsi que les notions de récursivité et la différence entre arguments positionnels et nommés.

B. Portée des variables

Az Définition

La portée d'une variable détermine où cette variable peut être utilisée dans le code. En Python, il existe deux types de portée : la portée globale et la portée locale.

Méthode

Variable globale : Une variable définie en dehors de toute fonction a une portée globale, ce qui signifie qu'elle peut être accédée et modifiée par n'importe quelle partie du programme.

```
1 variable_globale : str = "Je suis une variable globale"
2
3 def afficher_globale():
4     print(variable_globale)
5
6 afficher_globale() # Affiche "Je suis une variable globale"
```

Variable locale : une variable définie à l'intérieur d'une fonction a une portée locale, ce qui signifie qu'elle ne peut être utilisée qu'à l'intérieur de cette fonction.

```
1 def ma_fonction():
2     variable_locale: str = "Je suis une variable locale"
3     print(variable_locale)
4
5 ma_fonction() # Affiche "Je suis une variable locale"
6 # print(variable_locale)  # Provoque une erreur, car variable_locale n'existe
  pas en dehors de la fonction
```

Il est important de noter que si une variable locale a le même nom qu'une variable globale, la variable locale masquera la variable globale à l'intérieur de la fonction.

```
1 variable_globale = "Je suis globale"
2
3 def ma_fonction():
4     variable_globale = "Je suis locale"
5     print(variable_globale)
6
7 ma_fonction() # Affiche "Je suis locale"
8 print(variable_globale)  # Affiche "Je suis globale"
```

Tous droits réservés à STUDI - Reproduction interdite

C. Fonctions avec paramètres

Méthode

Les paramètres permettent de passer des valeurs à une fonction lors de son appel. Les fonctions peuvent avoir des paramètres obligatoires ou optionnels.

Les paramètres obligatoires sont ceux qui doivent être fournis lors de l'appel de la fonction. Reprenons notre méthode saluer.

```
1 def saluer(nom):
2     print(f"Bonjour, {nom}!")
3
4 saluer("Alice")  # Affiche "Bonjour, Alice!"
5 # saluer() # Provoque une erreur, car le paramètre 'nom' est obligatoire
```

Ici, nous avons mis nom en argument ; ainsi il est obligatoire, sinon, saluer() ne peut pas être correctement exécutée. À l'inverse, nous pouvons mettre une valeur par défaut à nom, ce qui rend nom optionnel.

```
1 def saluer(nom="invité"):
2     print(f"Bonjour, {nom}!")
3
4 saluer("Alice")  # Affiche "Bonjour, Alice!"
5 saluer() # Affiche "Bonjour, invité!"
```

D. Fonction récursive

Az Définition

Une fonction récursive est une fonction qui s'appelle elle-même. Les fonctions récursives sont souvent utilisées pour résoudre des problèmes qui peuvent être décomposés en sous-problèmes similaires.

Méthode

Par exemple, prenons la factorielle. La factorielle d'un nombre est l'ensemble de produits de tous les éléments entre lui et 1. Par exemple, la factorielle de 5, est 5*4*3*2*1 ce qui donne 120.

Une première approche serait de faire une boucle afin de calculer ce nombre :

```
1 def factorielle(nombre: int) -> int:
2     if nombre ==0:
3         return 1
4     resultat: int = 1
5     for i in range(1, nombre+1):
6         resultat = resultat * i
7     return(resultat)
8 print(factorielle(5))  # Affiche 120
```

Le code peut être simplifié via la récursivité. Nous allons depuis factorielle, rappeler factorielle tant que le nombre est supérieur à 1. Un fois que nous avons atteint 1, nous allons renvoyer le résultat.

Tous droits réservés à STUDI - Reproduction interdite

```python
def factorielle(nombre : int):
    if nombre == 0:
        return 1
    else:
        return nombre * factorielle(nombre-1)
```

Dans cet exemple, la fonction « *factorielle* » appelle elle-même avec une valeur réduite jusqu'à atteindre la base de la récursion (n == 0).

E. Arguments positionnels vs arguments nommés

Les arguments positionnels

Les arguments positionnels sont passés à la fonction **en fonction de leur position**. L'ordre dans lequel vous passez les arguments est important.

Méthode

Par exemple, j'ai une fonction qui a besoin de deux éléments, à savoir, nom et âge. Je dois d'abord préciser le nom, puis l'âge, comme ce que nous avons fait jusqu'à maintenant.

```python
def afficher_informations(nom, age):
    print(f"Nom: {nom}, Age: {age}")

afficher_informations("Alice", 30) # Affiche "Nom: Alice, Age: 30"
```

Les arguments nommés

Les arguments nommés sont passés à la fonction **en spécifiant le nom du paramètre**. L'ordre n'a pas d'importance avec les arguments nommés. Pour cela, nous pouvons préciser le champ lors de l'appel.

```python
def afficher_informations(nom, age):
    print(f"Nom: {nom}, Age: {age}")

afficher_informations(age=30, nom="Alice")  # Affiche "Nom: Alice, Age: 30"
```

Méthode

Les arguments nommés peuvent également être utilisés pour fournir des valeurs uniquement aux paramètres optionnels, en laissant les paramètres obligatoires comme arguments positionnels.

```python
def afficher_informations(nom, age=25):
    print(f"Nom: {nom}, Age: {age}")

afficher_informations("Alice", age=30) # Affiche "Nom: Alice, Age: 30"
afficher_informations("Bob")  # Affiche "Nom: Bob, Age: 25"
```

F. Exercice : Quiz

[solution n°2 p. 26]

Question 1

Quelle est la portée d'une variable définie à l'intérieur d'une fonction ?

Globale

Tous droits réservés à STUDI - Reproduction interdite

Locale

Semi-globale

Permanente

Question 2

Que se passe-t-il si une variable locale et une variable globale ont le même nom ?

La variable globale est toujours utilisée

La variable locale est toujours utilisée

Une erreur de syntaxe est levée

La variable globale est modifiée

Question 3

Quelle est la syntaxe correcte pour définir une fonction avec un paramètre optionnel « *nom* » ayant pour valeur par défaut "invité" ?

« *def saluer(nom: "invité"):* »

« *def saluer(nom="invité"):* »

« *def saluer(nom="invité": str):* »

« *def saluer(nom->"invité"):* »

Question 4

Qu'est-ce qu'une fonction récursive ?

Une fonction qui prend une autre fonction en paramètre

Une fonction qui s'appelle elle-même

Une fonction qui ne retourne aucune valeur

Une fonction définie à l'intérieur d'une autre fonction

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Quelle est la différence principale entre les arguments positionnels et les arguments nommés ?

- Les arguments positionnels doivent être des entiers, les arguments nommés peuvent être de n'importe quel type
- Les arguments positionnels sont passés selon leur position, les arguments nommés sont passés avec leur nom
- Les arguments positionnels peuvent avoir des valeurs par défaut, les arguments nommés ne peuvent pas
- Il n'y a pas de différence entre les arguments positionnels et nommés

Tous droits réservés à STUDI - Reproduction interdite

IV Fonctions lambda et expressions lambda

A. Introduction

Les fonctions lambda, aussi appelées expressions lambda, sont une manière concise de définir des fonctions en Python. Elles sont souvent utilisées pour des opérations simples et rapides, là où définir une fonction classique serait trop verbeux. Dans cette partie, nous allons explorer la syntaxe et l'utilisation des fonctions lambda, ainsi que les comparer aux fonctions classiques pour mieux comprendre leurs avantages et leurs limitations.

B. Définition et utilisation

Az Définition

Une fonction lambda en Python est une fonction anonyme, c'est-à-dire une fonction sans nom. Elle est définie à l'aide du mot-clé « *lambda* », suivi d'une liste de paramètres, d'un deux-points (`:`), et d'une expression. La valeur de l'expression est automatiquement retournée.

```
1 lambda arguments: expression
```

Voici un exemple d'une fonction lambda qui effectue une addition de deux nombres :

```
1 addition = lambda x, y: x + y
2 print(addition(5, 3))  # Affiche 8
```

Dans cet exemple, « *lambda x, y: x + y* » est une fonction lambda qui prend deux arguments, « *x* » et « *y* », et retourne leur somme.

Fondamental

Les fonctions lambda sont souvent passées en tant qu'arguments à des fonctions qui prennent d'autres fonctions comme paramètres. Par exemple, les fonctions « *map* », « *filter* », et « *sorted* » utilisent fréquemment des lambdas.

Exemple

José, le propriétaire de notre café, souhaite trier une liste de commandes selon le montant total de chaque commande. Chaque commande est représentée par un dictionnaire avec un « *nom* » et un « *montant* ».

```
1 commandes = [
2     {"nom": "café", "montant": 2.50},
3     {"nom": "sandwich", "montant": 5.00},
4     {"nom": "gâteau", "montant": 3.75}
5 ]
```

Nous allons utiliser une lambda afin de spécifier que le tri doit s'effectuer en fonction du montant de chaque commande. Cela permettra à José de connaître facilement lesquels, sont les plus coûteuses.

```
1 commandes_triees = sorted(commandes, key=lambda x: x["montant"], reverse=True)
2 print(commandes_triees)
```


Tous droits réservés à STUDI - Reproduction interdite

La lambda, qui s'appelle x, va prendre chaque élément du dictionnaire « *commandes* ». Nous allons trier par ordre croissant de montant des commandes, mais nous voulons en décroissant, donc il faut utiliser reverse=True afin d'inverser le rangement et procéder à un ordre décroissant.

Il est possible de fusionner les lambdas et les fonctions, pour ce faire, définissons une fonction en python.

```
def addition(x, y):
    return x + y
```

Maintenant, essayons d'en faire une lambda :

```
func = addition
print(func(5, 3))  # Affiche 8
```

En appelant func, j'exécute la fonction addition en lui fournissant les deux arguments afin de procéder à l'addition.

Fondamental

Les lambdas sont concises et écrites en une seule ligne, ce qui les rend idéales pour les fonctions simples et jetables. Les fonctions classiques, définies avec `def`, peuvent contenir plusieurs instructions, ce qui les rend plus adaptées aux opérations complexes. Les lambdas ne peuvent contenir qu'une seule expression et n'acceptent pas de blocs d'instructions, tandis que les fonctions classiques peuvent contenir plusieurs instructions et structures de contrôle, comme des boucles et des conditions.

C. Exercice : Quiz

[solution n°3 p. 28]

Question 1

Quelle est la syntaxe correcte pour définir une fonction lambda qui ajoute deux nombres ?

« *lambda: x, y -> x + y* »

« *lambda x y: x + y* »

« *lambda x, y: x + y* »

« *lambda (x, y): x + y* »

Question 2

Quelle est la sortie du code suivant ?

```
montants = [12, 5, 18, 7, 10]
remises = list(map(lambda montant: montant * 0.9 if montant > 10 else montant,
montants))
print(remises)
```

« *[10.8, 5, 16.2, 7, 9]* »

Tous droits réservés à STUDI - Reproduction interdite

« *[12, 5, 18, 7, 10]* »

« *[10.8, 5, 16.2, 7, 10]* »

« *[13.2, 5, 19.8, 7, 10]* »

Question 3

Quel est l'intérêt principal d'utiliser une fonction lambda dans une fonction de haut niveau telle que « *map* » ou « *filter* » ?

Les lambdas sont plus rapides que les fonctions classiques

Les lambdas peuvent contenir plusieurs instructions

Les lambdas sont plus concises et faciles à écrire pour des opérations simples

Les lambdas ne peuvent pas être utilisées dans « *map* » ou « *filter* »

Question 4

Quelle est la principale différence entre une fonction lambda et une fonction classique en Python ?

Les fonctions lambda peuvent contenir plusieurs expressions, les fonctions classiques ne peuvent en contenir qu'une seule

Les fonctions lambda sont anonymes, tandis que les fonctions classiques ont des noms

Les fonctions classiques sont toujours plus rapides que les lambdas

Les lambdas ne peuvent pas accepter de paramètres

Question 5

Quel est le résultat de l'exécution du code suivant ?

```python
commandes = [
    {"nom": "café", "montant": 2.50},
    {"nom": "sandwich", "montant": 5.00},
    {"nom": "gâteau", "montant": 3.75}
]

commandes_triees = sorted(commandes, key=lambda commande: commande["montant"])
print(commandes_triees)
```

) {"nom": "sandwich", {"nom": "gâteau", {"nom": "café",]`
`["montant": 5.00}, "montant": 3.75}, "montant": 2.50}

Tous droits réservés à STUDI - Reproduction interdite

```
`[ {"nom": "gâteau", "montant": 3.75}, {"nom": "café", "montant": 2.50}, {"nom": "sandwich", "montant": 5.00} ]`
```

```
`[ {"nom": "café", "montant": 2.50}, {"nom": "gâteau", "montant": 3.75}, {"nom": "sandwich", "montant": 5.00} ]`
```

```
`[ {"nom": "café", "montant": 2.50}, {"nom": "sandwich", "montant": 5.00}, {"nom": "gâteau", "montant": 3.75} ]`
```

Tous droits réservés à STUDI - Reproduction interdite

V Fonctions Génératrices et Itérateurs

A. Introduction

Les générateurs et les itérateurs sont des concepts puissants en Python qui permettent de gérer des séquences de données de manière efficace, sans avoir à charger toutes les données en mémoire. Les générateurs utilisent l'instruction « *yield* » pour produire des valeurs une à une, tandis que les itérateurs sont des objets qui implémentent les méthodes « *__iter__* » et « *__next__* ». Dans cette partie, nous allons explorer les fonctions génératrices, leur définition, leur utilisation, ainsi que les coroutines basées sur des générateurs, une nouveauté introduite dans Python 3.12.

B. Fonctions génératrices avec « yield »

Az Définition

Les fonctions génératrices sont un type spécial de fonction en Python qui utilise l'instruction « *yield* » pour produire une séquence de valeurs. Contrairement aux fonctions classiques, les générateurs ne retournent pas une valeur unique mais génèrent une série de valeurs au fil du temps.

Méthode

Une fonction génératrice est définie comme une fonction normale, mais utilise « *yield* » au lieu de « *return* » pour produire des valeurs.

```
1 def generate_numbers():
2     yield 1
3     yield 2
4     yield 3
```

Pour utiliser une fonction génératrice, vous pouvez l'appeler pour obtenir un objet générateur, puis utiliser une boucle « *for* » ou la fonction « *next()* » pour itérer sur les valeurs générées.

```
1 gen = generate_numbers()
2
3 # Utilisation avec une boucle for
4 for number in gen:
5     print(number)  # Affiche 1, puis 2, puis 3
6
7 # Utilisation avec next()
8 gen = generate_numbers()
9 print(next(gen))  # Affiche 1
10 print(next(gen))  # Affiche 2
11 print(next(gen))  # Affiche 3
12 # print(next(gen))  # Provoque une erreur StopIteration car il n'y a plus de
valeurs à générer
```

Les générateurs sont particulièrement utiles pour travailler avec des flux de données potentiellement infinis ou de grande taille, car ils génèrent les valeurs à la demande et utilisent la mémoire de manière efficace.

Voici un exemple pratique d'un générateur qui produit des nombres pairs jusqu'à une limite donnée.

Tous droits réservés à STUDI - Reproduction interdite

```python
def generate_even_numbers(limit):
    for i in range(limit):
        if i % 2 == 0:
            yield i

even_numbers = generate_even_numbers(10)
for number in even_numbers:
    print(number)  # Affiche 0, 2, 4, 6, 8
```

C. Nouveautés de Python 3.12 : Generator-based coroutines

Méthode

Python 3.12 introduit des améliorations aux coroutines basées sur des générateurs, rendant la gestion des tâches asynchrones encore plus flexible. Une coroutine basée sur des générateurs est une fonction génératrice qui peut utiliser « *yield* » pour suspendre son exécution et « *await* » pour attendre les résultats d'autres coroutines.

Les coroutines basées sur des générateurs sont définies avec « *async def* » et utilisent « *yield* » pour produire des valeurs.

```python
import asyncio

async def generator_coroutine():
    for i in range(3):
        await asyncio.sleep(1)
        yield i

# Utilisation d'une coroutine basée sur des générateurs
async def main():
    async for value in generator_coroutine():
        print(value)  # Affiche 0, puis 1, puis 2 avec un délai d'une seconde entre chaque

asyncio.run(main())
```

Tous droits réservés à STUDI - Reproduction interdite

VI Essentiel

Les fonctions jouent un rôle central dans la programmation en Python, offrant une manière de structurer, réutiliser et organiser le code de manière efficace.

Les fonctions permettent de regrouper des blocs de code sous un nom unique, facilitant ainsi la réutilisabilité, la modularité, la lisibilité et la maintenance du code. Elles sont définies avec le mot-clé `def` suivi du nom de la fonction, des parenthèses et d'un bloc de code indenté. L'instruction `return` permet de renvoyer une valeur depuis une fonction, et les fonctions peuvent renvoyer plusieurs valeurs en utilisant des tuples.

Les variables globales, définies en dehors des fonctions, sont accessibles partout dans le programme. En revanche, les variables locales, définies à l'intérieur des fonctions, ne sont accessibles qu'à l'intérieur de celles-ci. Les fonctions peuvent avoir des paramètres obligatoires ou optionnels avec des valeurs par défaut. Elles peuvent également être récursives, c'est-à-dire s'appeler elles-mêmes. Les arguments peuvent être passés positionnellement ou nommément, offrant ainsi une grande flexibilité dans la manière d'appeler les fonctions.

Les fonctions lambda, ou expressions lambda, sont des fonctions anonymes définies avec le mot-clé `lambda`. Elles sont concises et idéales pour des opérations simples, souvent utilisées comme arguments à des fonctions de haut niveau telles que `map`, `filter` et `sorted`. Bien que les lambdas soient moins lisibles et flexibles que les fonctions classiques, elles sont très utiles pour les tâches rapides et temporaires.

Les fonctions génératrices utilisent l'instruction `yield` pour produire une séquence de valeurs de manière paresseuse, ce qui permet de gérer des flux de données de manière efficace sans charger toutes les données en mémoire. Les générateurs sont particulièrement utiles pour travailler avec des séquences potentiellement infinies ou de grande taille. Une nouveauté de Python 3.12 est l'introduction des coroutines basées sur des générateurs, qui combinent les capacités des générateurs et des coroutines, permettant des opérations asynchrones plus flexibles. Les coroutines basées sur des générateurs sont définies avec `async def` et peuvent utiliser `yield` pour produire des valeurs ou déléguer à un autre générateur.

Tous droits réservés à STUDI - Reproduction interdite

VII Auto-évaluation

A. Exercice

Question 1

[solution n°4 p. 30]

Écrivez une fonction `multiplier` qui prend deux paramètres `a` et `b` et retourne leur produit. Appelez cette fonction avec les valeurs 5 et 6, puis affichez le

Question 2

[solution n°5 p. 30]

Considérez le code suivant et déterminez la sortie. Expliquez pourquoi.

```
x = 10

def changer_valeur():
    x = 20
    print(x)

changer_valeur()
print(x)

```

Question 3

[solution n°6 p. 30]

Utilisez une fonction lambda pour trier une liste de tuples représentant des produits, en fonction du prix.

```
produits = [("Pommes", 2.5), ("Bananes", 1.2), ("Cerises", 3.0)]
# Trier par prix croissant
produits_triees = []
print(produits_triees)
```

Question 4

[solution n°7 p. 30]

Écrivez une fonction génératrice `compteur` qui génère les nombres de 0 à 5. Utilisez une boucle `for` pour afficher chaque nombre généré.

B. Test

Exercice 1 : Quiz

[solution n°8 p. 30]

Question 1

Quelle est la sortie de ce code ?

```
def ajouter(a, b):
    return a + b

resultat = ajouter(3, 4)
print(resultat)
```

- 7
- 34
- 12
- Erreur

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Que se passe-t-il si une variable locale et une variable globale ont le même nom dans une fonction ?

- La variable globale est utilisée
- La variable locale est utilisée
- Une erreur de syntaxe est levée
- La variable globale est modifiée

Question 3

Quelle est la syntaxe correcte pour une fonction lambda qui multiplie deux nombres ?

- `lambda x, y: x * y`
- `lambda: x, y -> x * y`
- `lambda (x, y): x * y`
- `lambda x y: x * y`

Question 4

Quelle est la principale différence entre une fonction classique et une fonction lambda en Python ?

- Les fonctions classiques ne peuvent pas avoir de paramètres
- Les lambdas peuvent contenir plusieurs instructions
- Les lambdas sont anonymes, tandis que les fonctions classiques ont des noms
- Les fonctions classiques sont plus rapides que les lambdas

Question 5

Quel est le résultat de l'exécution du code suivant ?

```python
def generate_numbers():
    yield 1
    yield 2
    yield 3

gen = generate_numbers()
print(next(gen))
print(next(gen))
```


Tous droits réservés à STUDI - Reproduction interdite

```
9 print(next(gen))
10
```

1 2 3

3 2 1

Erreur

1 2 Erreur

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 7]

Question 1

Qu'est-ce qu'une fonction en programmation ?

- Un type de variable qui peut stocker des valeurs
- ✓ Un bloc de code qui effectue une tâche particulière et peut être appelé à tout moment
- Un type de boucle qui permet d'itérer sur des éléments
- Une erreur de syntaxe dans un programme

Une fonction est un ensemble d'instructions regroupées sous un nom unique qui peut être exécuté en appelant ce nom.

Question 2

Pourquoi est-il important d'utiliser des fonctions dans un programme ?

- Pour augmenter la taille du code source
- Pour rendre le code plus compliqué
- ✓ Pour améliorer la réutilisabilité, la modularité et la lisibilité du code
- Pour réduire la vitesse d'exécution du programme

Les fonctions permettent de structurer le code de manière claire, de le réutiliser facilement et de le rendre plus compréhensible.

Question 3

Quelle est la syntaxe correcte pour déclarer une fonction en Python qui s'appelle « *saluer* » et affiche "Bonjour" ?**

- « *def saluer: print("Bonjour")* »
- « *function saluer() print("Bonjour")* »

Tous droits réservés à STUDI - Reproduction interdite

✓ « *def saluer(): print("Bonjour")* »

« *saluer() { print("Bonjour") }* »

En Python, une fonction est déclarée avec le mot-clé « *def* », suivi du nom de la fonction et de parenthèses. Le bloc de code de la fonction est indenté.

Question 4

Que fait l'instruction « *return* » dans une fonction ?

Elle arrête l'exécution du programme

Elle imprime une valeur sur la console

✓ Elle renvoie une valeur depuis la fonction vers l'appelant

Elle initialise une variable

L'instruction « *return* » permet de sortir d'une fonction et de renvoyer une valeur à l'endroit où la fonction a été appelée.

Question 5

Quelle est la sortie du code suivant ?

```
1 def additionner(a, b):
2     return a + b
3
4 resultat = additionner(3, 4)
5 print(resultat)
```

3

4

✓ 7

12

La fonction « *additionner* » prend deux paramètres « *a* » et « *b* », et renvoie leur somme. Quand elle est appelée avec les arguments 3 et 4, elle renvoie 7.

Tous droits réservés à STUDI - Reproduction interdite

studi

Solution n°2

[exercice p. 11]

Question 1

Quelle est la portée d'une variable définie à l'intérieur d'une fonction ?

- Globale
- ✓ Locale
- Semi-globale
- Permanente

Une variable définie à l'intérieur d'une fonction est locale à cette fonction et ne peut être utilisée qu'à l'intérieur de celle-ci.

Question 2

Que se passe-t-il si une variable locale et une variable globale ont le même nom ?

- La variable globale est toujours utilisée
- ✓ La variable locale est toujours utilisée
- Une erreur de syntaxe est levée
- La variable globale est modifiée

La variable locale masque la variable globale à l'intérieur de la fonction.

Question 3

Quelle est la syntaxe correcte pour définir une fonction avec un paramètre optionnel « *nom* » ayant pour valeur par défaut "invité" ?

- « *def saluer(nom: "invité"):* »
- ✓ « *def saluer(nom="invité"):* »
- « *def saluer(nom="invité": str):* »
- « *def saluer(nom->"invité"):* »

Tous droits réservés à STUDI - Reproduction interdite

La syntaxe correcte pour définir une fonction avec un paramètre optionnel en Python consiste à assigner une valeur par défaut au paramètre dans la déclaration de la fonction.

Question 4

Qu'est-ce qu'une fonction récursive ?

- Une fonction qui prend une autre fonction en paramètre
- ✓ Une fonction qui s'appelle elle-même
- Une fonction qui ne retourne aucune valeur
- Une fonction définie à l'intérieur d'une autre fonction

Une fonction récursive est une fonction qui s'appelle elle-même pour résoudre un problème en le décomposant en sous-problèmes similaires.

Question 5

Quelle est la différence principale entre les arguments positionnels et les arguments nommés ?

- Les arguments positionnels doivent être des entiers, les arguments nommés peuvent être de n'importe quel type
- ✓ Les arguments positionnels sont passés selon leur position, les arguments nommés sont passés avec leur nom
- Les arguments positionnels peuvent avoir des valeurs par défaut, les arguments nommés ne peuvent pas
- Il n'y a pas de différence entre les arguments positionnels et nommés

Les arguments positionnels sont assignés en fonction de leur ordre dans la liste des paramètres, tandis que les arguments nommés sont assignés en fonction du nom des paramètres, indépendamment de leur position.

Tous droits réservés à STUDI - Reproduction interdite

studi

Solution n°3

[exercice p. 15]

Question 1

Quelle est la syntaxe correcte pour définir une fonction lambda qui ajoute deux nombres ?

« *lambda: x, y -> x + y* »

« *lambda x y: x + y* »

✓ « *lambda x, y: x + y* »

« *lambda (x, y): x + y* »

La syntaxe correcte pour définir une fonction lambda en Python est « *lambda* » suivi des paramètres séparés par des virgules, un deux-points, et l'expression qui retourne la valeur.

Question 2

Quelle est la sortie du code suivant ?

```
1 montants = [12, 5, 18, 7, 10]
2 remises = list(map(lambda montant: montant * 0.9 if montant > 10 else montant,
  montants))
3 print(remises)
```

« *[10.8, 5, 16.2, 7, 9]* »

« *[12, 5, 18, 7, 10]* »

✓ « *[10.8, 5, 16.2, 7, 10]* »

« *[13.2, 5, 19.8, 7, 10]* »

La fonction lambda applique une remise de 10 % aux montants supérieurs à 10, laissant les autres montants inchangés. Les montants 12 et 18 sont réduits à 10.8 et 16.2 respectivement.

Question 3

Quel est l'intérêt principal d'utiliser une fonction lambda dans une fonction de haut niveau telle que « *map* » ou « *filter* » ?

Les lambdas sont plus rapides que les fonctions classiques

Les lambdas peuvent contenir plusieurs instructions

Tous droits réservés à STUDI - Reproduction interdite

✓ Les lambdas sont plus concises et faciles à écrire pour des opérations simples

Les lambdas ne peuvent pas être utilisées dans « *map* » ou « *filter* »

Les fonctions lambda sont souvent utilisées pour des opérations simples et rapides, là où définir une fonction classique serait trop verbeux.

Question 4

Quelle est la principale différence entre une fonction lambda et une fonction classique en Python ?

Les fonctions lambda peuvent contenir plusieurs expressions, les fonctions classiques ne peuvent en contenir qu'une seule

✓ Les fonctions lambda sont anonymes, tandis que les fonctions classiques ont des noms

Les fonctions classiques sont toujours plus rapides que les lambdas

Les lambdas ne peuvent pas accepter de paramètres

Les lambdas sont des fonctions anonymes, définies sans nom, contrairement aux fonctions classiques qui sont définies avec un nom.

Question 5

Quel est le résultat de l'exécution du code suivant ?

```python
commandes = [
    {"nom": "café", "montant": 2.50},
    {"nom": "sandwich", "montant": 5.00},
    {"nom": "gâteau", "montant": 3.75}
]

commandes_triees = sorted(commandes, key=lambda commande: commande["montant"])
print(commandes_triees)
```

) {"nom": "sandwich", {"nom": "gâteau", {"nom": "café",]`
`["montant": 5.00}, "montant": 3.75}, "montant": 2.50}

`[ {"nom": "gâteau", {"nom": "café", "montant": {"nom": "sandwich", ]`
"montant": 3.75}, 2.50}, "montant": 5.00}

✓ `[ {"nom": "café", {"nom": "gâteau", {"nom": "sandwich", ]`
"montant": 2.50}, "montant": 3.75}, "montant": 5.00}

Tous droits réservés à STUDI - Reproduction interdite

`[ {"nom": "café", "montant": 2.50}, {"nom": "sandwich", "montant": 5.00}, {"nom": "gâteau", "montant": 3.75} ]`

La fonction lambda utilisée dans « *sorted* » trie les commandes en fonction du montant en ordre croissant. Les montants de 2.50, 3.75, et 5.00 sont triés du plus petit au plus grand.

[exercice p. 21]

Solution n°4

```
def multiplier(a, b):
    return a * b

resultat = multiplier(5, 6)
print(resultat)  # Affiche 30
```

[exercice p. 21]

Solution n°5

```
20
10
```

La variable x à l'intérieur de la fonction `changer_valeur` est locale et masque la variable globale x à l'intérieur de la fonction.

[exercice p. 21]

Solution n°6

```
produits = [("Pommes", 2.5), ("Bananes", 1.2), ("Cerises", 3.0)]
produits_triees = sorted(produits, key=lambda produit: produit[1])
print(produits_triees)
# Affiche [("Bananes", 1.2), ("Pommes", 2.5), ("Cerises", 3.0)]
```

[exercice p. 21]

Solution n°7

```
def compteur():
    for i in range(6):
        yield i

for nombre in compteur():
    print(nombre)
# Affiche 0, 1, 2, 3, 4, 5
```

Solution n°8

[exercice p. 21]

Question 1

Quelle est la sortie de ce code ?

```
def ajouter(a, b):
    return a + b

resultat = ajouter(3, 4)
print(resultat)
```


Tous droits réservés à STUDI - Reproduction interdite

✓ 7

34

12

Erreur

On additionne 3 avec 4, ce qui donne 7.

Question 2

Que se passe-t-il si une variable locale et une variable globale ont le même nom dans une fonction ?

La variable globale est utilisée

✓ La variable locale est utilisée

Une erreur de syntaxe est levée

La variable globale est modifiée

Une variable locale dans une fonction, écrase la valeur de la valeur globale si elle porte le même nom

Question 3

Quelle est la syntaxe correcte pour une fonction lambda qui multiplie deux nombres ?

✓ `lambda x, y: x * y`

`lambda: x, y -> x * y`

`lambda (x, y): x * y`

`lambda x y: x * y`

On détermine d'abord les éléments nécessaires, ici, x et y, puis on effectue une opération de multiplication.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quelle est la principale différence entre une fonction classique et une fonction lambda en Python ?

- Les fonctions classiques ne peuvent pas avoir de paramètres
- Les lambdas peuvent contenir plusieurs instructions
- ✓ Les lambdas sont anonymes, tandis que les fonctions classiques ont des noms
- Les fonctions classiques sont plus rapides que les lambdas

Les lambdas, n'ont pas de nom. Elles peuvent faire la même chose que les fonctions.

Question 5

Quel est le résultat de l'exécution du code suivant ?

```python
def generate_numbers():
    yield 1
    yield 2
    yield 3

gen = generate_numbers()
print(next(gen))
print(next(gen))
print(next(gen))

```

- ✓ 1 2 3
- 3 2 1
- Erreur
- 1 2 Erreur

On reçoit d'abord 1, via "yield 1", puis 2, puis 3.

Tous droits réservés à STUDI - Reproduction interdite